

ZJUI 学院 2021 年暑期科研项目总结材料



中文论文题目： 基于时空域反向传播的复用 LIF 方法

英文论文题目： **A Multiplexed Approach For Leaky Integrate
and Fire Model Based On Spatial Temporal
Back Propagation**

成员姓名： 邬飞扬（3210110569）

何俊辰（学号）

蔡胜南（学号）

指导教师： 金潮渊

指导教师所在学院： 信息与电子工程学院

暑期研究起止日期： 2024 年 06 月 17 日-2024 年 07 月 17 日

Abstract

This paper introduces a novel multiplexed approach for the Leaky Integrate-and-Fire (LIF) model, leveraging SpatioTemporal Back Propagation (STBP) to enhance the performance of Spiking Neural Networks (SNNs). The LIF model, a fundamental framework simulating neuronal dynamics, is augmented with a time division multiplexing technique that allows for the efficient handling of multiple input channels through a single LIF unit. This method, termed Time Division Alternating Multiplexed LIF (MLIF), alternates the input channels at each time step, preserving the distinct firing rates of the original channels. The proposed approach is inspired by the multiplexing phenomena observed in biological neural systems and optical neural networks (ONNs), aiming to reduce the number of physical components required in SNN hardware implementations. The MLIF model is integrated with the STBP algorithm, which is adept at handling the backpropagation of errors through the network while accounting for the temporal dynamics of spiking neurons. Through simulations and experiments on the MNIST dataset, the paper demonstrates that the Alternating Multiplexed LIF architecture can achieve high accuracy while maintaining the efficiency of information transfer. This research contributes to the understanding of how SNNs can mimic the temporal properties of the human brain by multiplexing different neural codes within the data-paths of actual neurons, offering a promising direction for the development of advanced neural network architectures.

1 Introduction

Spiking Neural Networks(SNNs) is a type of neural network that simulates the process of pulse issuance and information transfer in biological neurons. In SNN, neurons transmit information by issuing pulses.

Over the next seventy years, scientists have proposed a variety of models and learning methods for SNNs, which is to describe the information that is transferred and communicated between neurons. These include Integrate-and-fire and its various derivatives (e.g., LIF (leaky integrate-and-fire)), STDP (Spike-Timing-Dependent Plasticity)[4], STBP (Spike-based Backpropagation)[5] etc.

The Leaky Integrate-and-Fire (LIF) model is a widely used framework for simulating neuron dynamics in spiking neural networks (SNNs). This model simplifies neuron behavior by assuming that a neuron generates a spike when its membrane potential reaches a threshold value, after which the potential resets to a baseline level. The dynamics of the LIF model can be approximated as an RC circuit, capturing key neuronal processes such as charge accumulation, leakage, and spiking [1].

A neural network consists of three parts: an input layer, a hidden layer, and an output layer. Data enters through the input layer, undergoes a linear transformation using weights and biases, passes through an activation function, and then reaches the hidden layer, which serves as the input for the next layer. This process is known as forward propagation.

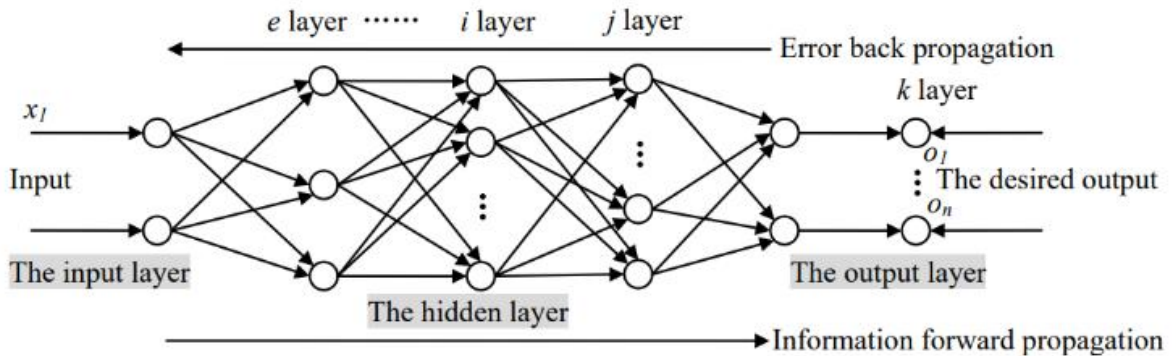


Fig.1. BP neural network model

The backpropagation (BP) algorithm uses gradient descent to minimize the loss function by adjusting the weights and biases. It calculates the gradients (partial derivatives) of the loss function with respect to the weights and biases and updates them iteratively until the loss

function reaches a minimum value or a set number of iterations is completed. This process finds the optimal parameters for the neural network [3].

Spike Timing Dependent Plasticity (STDP)[4] is a learning rule applied in SNNs where the strength of synaptic connections is adjusted based on the relative timing of pre- and post-synaptic spikes. Specifically, if a presynaptic spike precedes a postsynaptic spike, the connection strengthens, while if it follows, the connection weakens. This mechanism enables the network to learn temporal patterns and adjust synaptic weights accordingly.

Our research builds on the STDP algorithm to enhance two-channel SNN multiplexing networks. We employ the Spatio-Temporal Backpropagation (STBP) method, which integrates spatial and temporal aspects to model neuron layers over time. This method adjusts membrane potentials based on previous spikes and accumulates outputs from preceding neurons, thereby facilitating propagation through both spatial and temporal dimensions. Additionally, the STBP algorithm approximates the Dirac function, enriching the backpropagation architecture of SNNs.

Yifeng Liu's research [5] explores crosstalk and multiplexing in optical neural networks (ONNs) using a multi-neuron system (MNS) with wavelength division multiplexing (WDM) technology [6]. This method allows multiple inputs to be processed in parallel but can suffer from crosstalk between channels, reducing accuracy. To solve this, the paper proposes the NewBP algorithm, which adjusts traditional backpropagation to account for crosstalk, improving performance. Inspired by dual-channel multiplexing in ONNs, we plan to innovate in spiking neural networks (SNNs). Similarly, biological studies [2] show that neurons use multiplexed temporal scales to process information efficiently, suggesting a parallel approach for time-scale multiplexing.

Therefore, inspired by the success of dual-channel multiplexing in the ONN and biological findings, we try to make innovations on SNNs and we are inspired of the idea of parity multiplexing on time scales.

2 Related Work

2.1 Iterative Leaky Integrate-and-Fire Model

The equation for the original LIF:

$$x_i^{t+1,n} = \sum_{j=1}^{l(n-1)} w_{ij}^n o_j^{t+1,n-1} \quad (1)$$

$$u_i^{t+1,n} = u_i^{t,n} \tau e^{-\frac{o_i^{t,n}}{\tau}} + x_i^{t+1,n} + b_i^n \quad (2)$$

$$o_i^{t+1,n} = f(u_i^{t+1,n}) \quad (3)$$

Where:

$$f(x) = \begin{cases} 1 & x \geq u_{th} \\ 0 & x < u_{th} \end{cases} \quad (4)$$

$$(5)$$

Or we can rewrite this into matrix form:

$$\mathbf{u}^{t+1,n} = e^{-\frac{dt}{\tau}} \mathbf{u}^{t,n} \circ (1 - \mathbf{o}^{t,n}) \quad (6)$$

$$\mathbf{o}^{t+1,n} = f(\mathbf{u}^{t+1,n} - \mathbf{u}_{th}) \quad (7)$$

2.2 Spatial Temporal Back Propagation (STBP)

Spatial Temporal Back Propagation is one of the most important and advanced back propagation techniques, proposed by Wu et al [5].

The loss function of STBP is defined as:

$$L = \frac{1}{2S} \sum_{s=1}^S \|\mathbf{y}_s - \frac{1}{T} \sum_{t=1}^T \mathbf{o}_s^{t,n}\|_2^2 \quad (8)$$

We have:

$$\frac{\partial L}{\partial \mathbf{o}^{t,n}} = \frac{\partial L}{\partial \mathbf{o}^{t,n+1}} \frac{\partial \mathbf{o}^{t,n+1}}{\partial \mathbf{o}^{t,n}} + \frac{\partial L}{\partial \mathbf{o}^{t+1,n}} \frac{\partial \mathbf{o}^{t+1,n}}{\partial \mathbf{o}^{t,n}} \quad (9)$$

$$= (\mathbf{W}^{n+1})^T \frac{\partial L}{\partial \mathbf{u}^{t,n+1}} - e^{-\frac{dt}{\tau}} \frac{\partial L}{\partial \mathbf{u}^{t+1,n}} \circ (1 - \mathbf{o}^{t,n}) \quad (10)$$

$$\frac{\partial L}{\partial \mathbf{u}^{t,n}} = \frac{\partial L}{\partial \mathbf{o}^{t,n}} \frac{\partial \mathbf{o}^{t,n}}{\partial \mathbf{u}^{t,n}} + \frac{\partial L}{\partial \mathbf{o}^{t+1,n}} \frac{\partial \mathbf{o}^{t+1,n}}{\partial \mathbf{u}^{t,n}} \quad (11)$$

$$= \frac{\partial L}{\partial \mathbf{o}^{t,n}} \circ f' + e^{-\frac{dt}{\tau}} \frac{\partial L}{\partial \mathbf{u}^{t+1,n}} \circ (1 - \mathbf{o}^{t,n}) \quad (12)$$

The gradient of the weight can be written as:

$$\frac{\partial L}{\partial \mathbf{b}^n} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{u}^{t,n}} \quad (13)$$

$$\frac{\partial L}{\partial \mathbf{W}^n} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{u}^{t,n}} \mathbf{o}^{t,n-1T} \quad (14)$$

3 Method

3.1 Time Division Alternating Multiplexed LIF

Consider two channels of sum of the weighted spike ready to be the input of LIF module. We assume that these two channels take uses a single LIF. A simple and explicit way of doing this is to alternate the channel that is transmitting the spikes to the LIF at each time stamp during the timing window. We can call it the encoding stage.

The dynamics of LIF can be defined as:

$$\tau \frac{du}{dt} = -u(t) + I(t) \quad (15)$$

By solving the ODE and rewrite it as an iterative version we can get:

$$u(t_2) = u(t_1)e^{\frac{t_1-t_2}{\tau}} + I(t) \quad (16)$$

Let's say we have a time window with $T=20$, and two channels are fed into the same LIF. The concept of feeding alternate channels, say channel 1 and channel 2, by time, can be the case that when time index t is even, we take the spike on channel 1 and ignore the spike on channel 2, and when time t is odd, we take the spike on channel 2 and feed it into the LIF module. The LIF must output some kind of spike pattern, while we simply split the LIF output, once again into two channels, which is the decoding stage. When t is even, the output of the LIF is the output of channel 1 while channel 2 will be padded with 0 (no spike). And when t is odd the output of the LIF is the output of channel 2 while channel 1 will be padded with 0 (no spike).

As illustrated in Fig. 2, the red mask filters out the spikes on the first input channel with even time indices and the blue mask filters out spikes with odd time indices on the second channel, representing the encoding stage. Fig. 2. represent the spikes of the LIF output and the red mask filters out the spikes with even indices and load them to the first output channel while the blue mask filters out the spikes with even indices and load them to output channel two. The

odd time indices of output channel one and even indices of output channel two are padded with no spikes, representing the decoding stage.

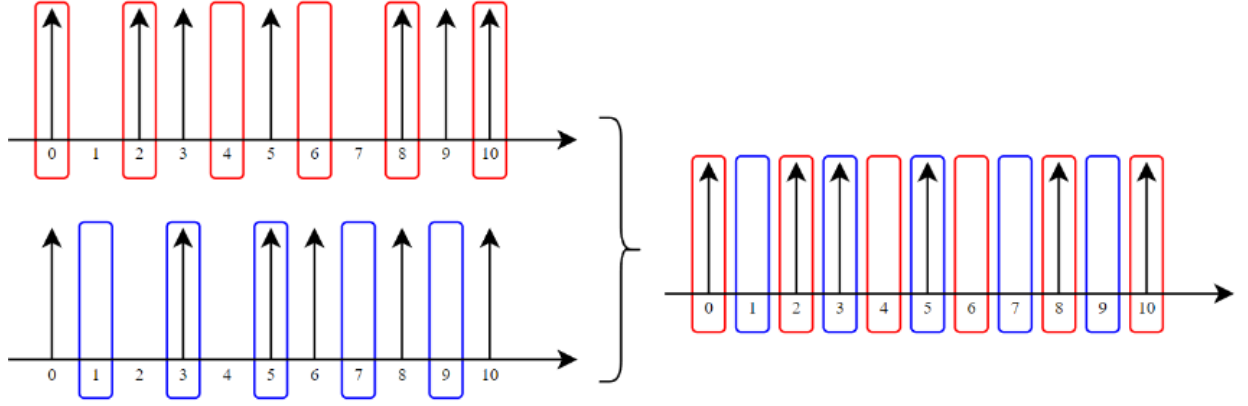


Fig.2. Encoding Stage

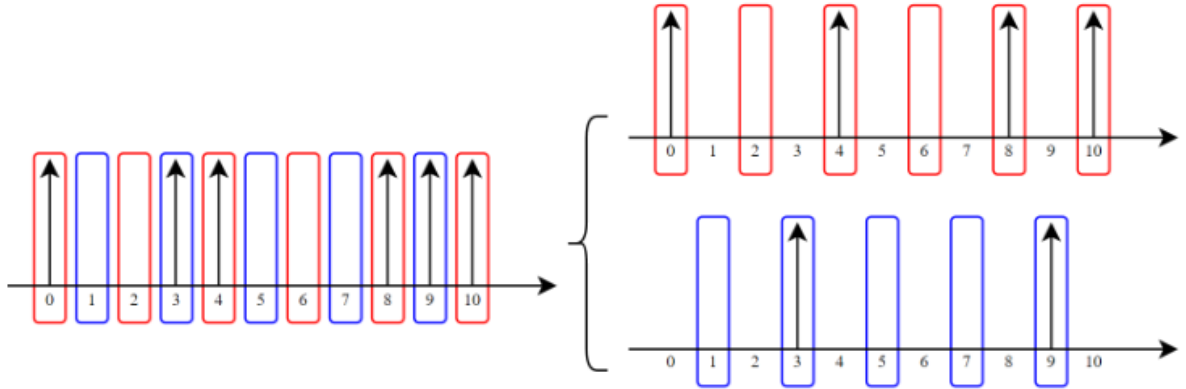


Fig.3. Encoding Stage

There are abundant similarities between the ONN mentioned earlier with the current approach. First, optical device is used to multiplex different channels into only one such channel using frequency property of light, and the optical device, the SOA, serves as the non-linear activation function of the neural network, while our method multiplexed two channels into one channel that go through a single LIF unit.

Now we have define the concept of Time Division Alternating Multiplexed Leaky-Integrate and Fire unit (MLIF).

3.2 Backpropagation technique for TDM Channel Alternating

Multiplexed LIF

Say we have weighted input matrix $\mathbf{u}^{t,n}$, the output of layer n and time stamp t , is a matrix with size $\text{batch_size} \times l(n)$, where $l(n)$ is the number of nodes within layer n . Because we use a 2-1 multiplexer, so we only have $l(n)/2$ nodes, so the size of the membrane potential matrix $\mathbf{u}^{t,n}$ will be $\text{batch_size} \times l(n)/2$.

At even index time step, we should only take the columns of the even indices (starting from 0). Actually we just need to apply a matrix $\mathbf{M}$ to the weighted input matrix so that we can feed into LIF:

$$M_{\text{even}} = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \\ 0 & 0 & 0 & \dots & 0 \end{bmatrix} \quad (17)$$

$$M_{\text{odd}} = \begin{bmatrix} 0 & 0 & 0 & \dots & 0 \\ 1 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix} \quad (18)$$

$$(19)$$

And the LIF input matrix should have the same shape as membrane potential matrix:

$$\mathbf{u}^{t,n} = \text{Membrane_update}(\mathbf{NM}) \quad (20)$$

The output of the LIF have the same shape as $\mathbf{u}^{t,n}$. To split that according to the odd or even channel, we can simply apply the transform of $\mathbf{M}$:

$$\mathbf{o}_{\text{odd}}^{t,n} = \mathbf{o}^{t,n} \mathbf{M}_{\text{odd}}^T \quad (21)$$

$$\mathbf{o}_{\text{even}}^{t,n} = \mathbf{o}^{t,n} \mathbf{M}_{\text{even}}^T \quad (22)$$

Using this method, we can now update the original LIF updating rule into the formulas (even indices):

$$\mathbf{u}^{t+1,n} = e^{-\frac{dt}{\tau}} \mathbf{u}^{t,n} \circ (1 - \mathbf{o}^{t,n} \mathbf{M}_{\text{even}}^T) \quad (23)$$

$$\mathbf{o}^{t+1,n} = f(\mathbf{u}^{t+1,n} - \mathbf{u}_{th}) \mathbf{M}_{\text{even}}^T \quad (24)$$

For $t = 2k, k \in \mathbb{N}$ we modify the LIF updating rule:

$$\mathbf{u}^{t+1,n} = e^{-\frac{dt}{\tau}} \mathbf{u}^{t,n} \circ (1 - \mathbf{o}^{t,n} M_{even}^T) \quad (25)$$

$$\mathbf{o}^{t+1,n} = f(\mathbf{u}^{t+1,n} - \mathbf{u}_{th}) M_{even}^T \quad (26)$$

For $t = 2k + 1, k \in \mathbb{N}$ we modify the LIF updating rule:

$$\mathbf{u}^{t+1,n} = e^{-\frac{dt}{\tau}} \mathbf{u}^{t,n} \circ (1 - \mathbf{o}^{t,n} M_{odd}^T) \quad (27)$$

$$\mathbf{o}^{t+1,n} = f(\mathbf{u}^{t+1,n} - \mathbf{u}_{th}) M_{odd}^T \quad (28)$$

After modifying the LIF updating rule, we can update the STBP rule for $t = 2k, k \in \mathbb{N}$:

$$\frac{\partial L}{\partial \mathbf{u}^{t,n}} = \frac{\partial L}{\partial \mathbf{o}^{t,n}} \frac{\partial \mathbf{o}^{t,n}}{\partial \mathbf{u}^{t,n}} + \frac{\partial L}{\partial \mathbf{u}^{t+1,n}} \frac{\partial \mathbf{u}^{t+1,n}}{\partial \mathbf{u}^{t,n}} \quad (29)$$

$$= \frac{\partial L}{\partial \mathbf{o}^{t,n}} \mathbf{M}_{even}^T \circ f' + \frac{\partial L}{\partial \mathbf{u}^{t+1,n}} e^{-\frac{dt}{\tau}} \circ (1 - \mathbf{o}^{t-1,n} \mathbf{M}_{even}) \quad (30)$$

$$\frac{\partial L}{\partial \mathbf{o}^{t,n}} = \frac{\partial L}{\partial \mathbf{o}^{t,n+1}} \frac{\partial \mathbf{o}^{t,n+1}}{\partial \mathbf{o}^{t,n}} + \frac{\partial L}{\partial \mathbf{o}^{t+1,n}} \frac{\partial \mathbf{o}^{t+1,n}}{\partial \mathbf{o}^{t,n}} \quad (31)$$

$$= (\mathbf{W}^{n+1})^T \mathbf{M}_{even} \frac{\partial L}{\partial \mathbf{u}^{t,n+1}} \quad (32)$$

$$- e^{-\frac{dt}{\tau}} \frac{\partial L}{\partial \mathbf{u}^{t+1,n}} \circ (1 - \mathbf{o}^{t,n} \mathbf{M}_{even}) \quad (33)$$

And for $t = 2k + 1, k \in \mathbb{N}$:

$$\frac{\partial L}{\partial \mathbf{u}^{t,n}} = \frac{\partial L}{\partial \mathbf{o}^{t,n}} \frac{\partial \mathbf{o}^{t,n}}{\partial \mathbf{u}^{t,n}} + \frac{\partial L}{\partial \mathbf{u}^{t+1,n}} \frac{\partial \mathbf{u}^{t+1,n}}{\partial \mathbf{u}^{t,n}} \quad (34)$$

$$= \frac{\partial L}{\partial \mathbf{o}^{t,n}} \mathbf{M}_{odd}^T \circ f' + \frac{\partial L}{\partial \mathbf{u}^{t+1,n}} e^{-\frac{dt}{\tau}} \circ (1 - \mathbf{o}^{t-1,n} \mathbf{M}_{odd}) \quad (35)$$

$$\frac{\partial L}{\partial \mathbf{o}^{t,n}} = \frac{\partial L}{\partial \mathbf{o}^{t,n+1}} \frac{\partial \mathbf{o}^{t,n+1}}{\partial \mathbf{o}^{t,n}} + \frac{\partial L}{\partial \mathbf{o}^{t+1,n}} \frac{\partial \mathbf{o}^{t+1,n}}{\partial \mathbf{o}^{t,n}} \quad (36)$$

$$= (\mathbf{W}^{n+1})^T \mathbf{M}_{odd} \frac{\partial L}{\partial \mathbf{u}^{t,n+1}} \quad (37)$$

$$- e^{-\frac{dt}{\tau}} \frac{\partial L}{\partial \mathbf{u}^{t+1,n}} \circ (1 - \mathbf{o}^{t,n} \mathbf{M}_{odd}) \quad (38)$$

We can use the above rules, along with the original STBP algorithm, to effectively use MLIF in the SNN network.

4 Experiment

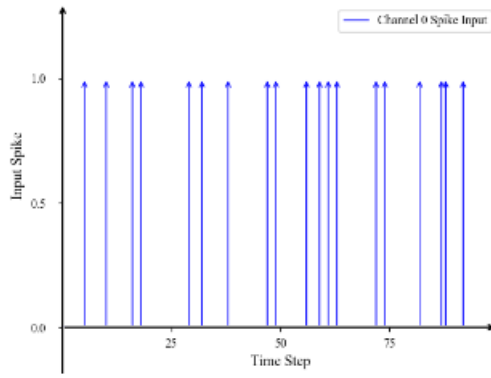
4.1 Alternating Multiplexed LIF Simulation

After summing the weighted spikes, the value is fed into the LIF unit the output is decoded into two channels. If no multiplexing method is utilized, it is obvious that a channel with relatively high rate will also get a output channel with relatively high rate while low rate channel will get a low rate output. It is very important that this pattern is preserved after two channels with distinct rate are multiplexed.

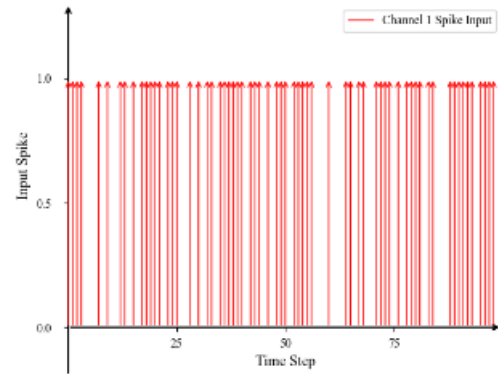
A simulation is done over the alternating multiplexed LIF, with total time window 200, decay factor $\tau = 0.95$, and threshold 4.

We first define channel 0 and channel 1, using Rate Encoding with Bernoulli Distribution firing on each time step. At each time step, channel 0 has the possibility of 0.2 to fire a spike and channel a possibility of 0.6 to fire a spike, as shown in Figure 4(a) and 4(b). We assume that the weights for both channels are 1 for simplicity.

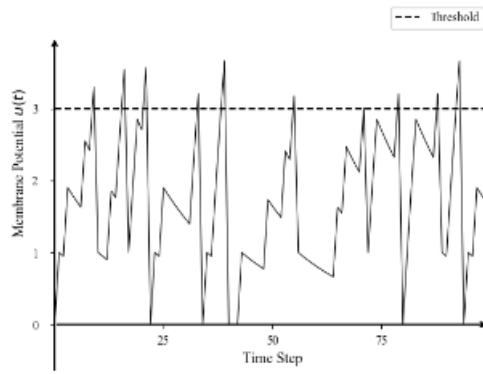
The membrane potential of the Alternating Multiplexed LIF, as shown in Figure 4(c), reflect the change in membrane potential for the alternating channel input. The output spikes, drawn in blue and red in Figure 4(d) representing channel 0 and channel 1 output, can correctly reflect the relative scarcity of the input. By the simulation process we can confirm that this alternating multiplexing approach can preserve the spike scarcity, or the rate which carries information.



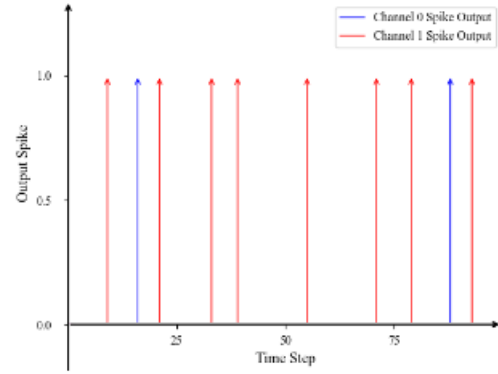
(a) Channel 0 Input



(b) Channel 1 input



(c) LIF Membrane Potential



(d) Channel 0 and 1 Output Spike

Figure. 4. Alternating Multiplexed LIF Simulation

4.2 Alternating Multiplexed LIF Simulation

If we fully implement Liu's [6] network, with simple structure while achieve relatively high performance on MNIST dataset, and a method of multiplexing two channels together.

By slightly modify the original nueral network we can get the nueral network structure as illustrated in Fig. 5.

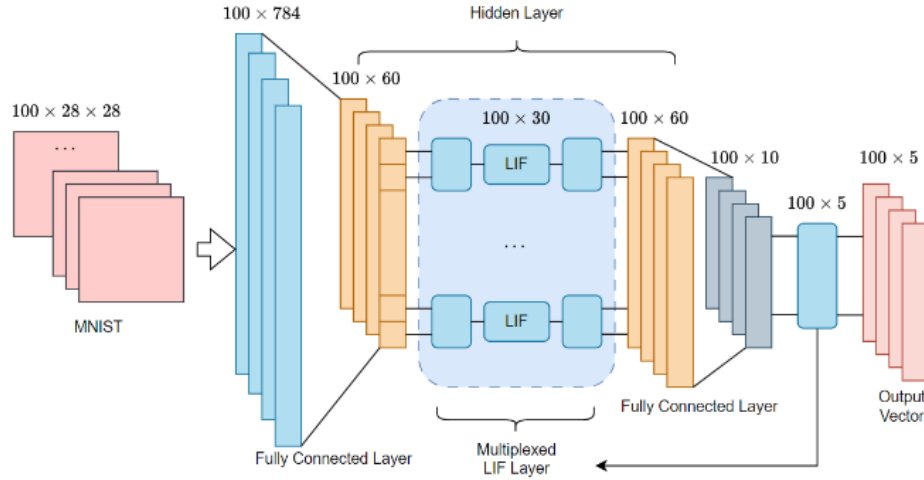


Fig. 5. Network Structure

First the MNIST image is unfolded into an array of 784 pixels. The image is fed into the network with batch size 100. The hidden layer has 60 nodes, and they were multiplexed into 30 spike dataflow paths before inputting into LIF unit. The LIFs output are decoded into 60 dataflow paths once again. The output layer has 10 nodes matching the total categories of MNIST dataset. Again, there 5 Alternating Multiplexed LIF following the fully connected layer after the hidden layer. The decoded LIF output is the final output for the network.

4.3 Result

Threshold	Decay	Batch Size	Learning Rate	Epoch	Training Accuracy	Testing Accuracy
0.5	0.2	100	1e-3	100	98%	96.79%

Table. 1.

The model can achieve relatively high accuracy after implementing the Alternating Multiplexed LIF architecture.

Conclusion

This work is inspired by the multiplexing phenomena of different neural codes functioning in actual brains, the multiplexing method in Optical Neural Network, and the original STBP algorithm.

The contribution of this work includes the following:

(1) *A temporal division multiplexing method*. This is a possible approach to multiplex two or several binary spiking signal into one channel.

(2) *A Back propagation friendly approach to multiplex two channels*. Introducing the even and odd indices selecting matrix and applying matrix multiplication method is easy to method that leads to easy computation of gradients during the backward propagation stage.

Our work will also show its effectiveness in terms of incorporating physical LIF hardware in the SNN which saves the number of these actual physical components. This research also reveals that, SNN's temporal property leaves room for mimicking human brains in terms of multiplexing different neural codes within data-paths of actual neurons.

References:

- [1] N. M. Samardzic, J. S. Bajic, D. L. Sekulic, and S. Dautovic, "Volatile Memristor in Leaky Integrate-and-Fire Neurons: Circuit Simulation and Experimental Study," *Electronics*, vol. 11, no. 6, Art. no. 6, Jan. 2022, doi: 10.3390/electronics11060894.
- [2] S. Panzeri, N. Brunel, N. K. Logothetis, and C. Kayser, "Sensory neural codes using multiplexed temporal scales," *Trends in Neurosciences*, vol. 33, no. 3, pp. 111–120, Mar. 2010, doi: 10.1016/j.tins.2009.12.001.
- [3] Zhao Y., "Research and Application on BP Neural Network Algorithm," presented at the 2015 International Industrial Informatics and Computer Engineering Conference, Atlantis Press, Mar. 2015, pp. 1444–1447. doi: 10.2991/iiicec-15.2015.321.
- [4] Y. Dan and M. Poo, "Spike Timing-Dependent Plasticity of Neural Circuits," *Neuron*, vol. 44, no. 1, pp. 23–30, Sep. 2004, doi: 10.1016/j.neuron.2004.09.007.
- [5] Wu, Yujie, Lei Deng, Guoqi Li, Jun Zhu, and Luping Shi. "Spatio-temporal backpropagation for training high-performance spiking neural networks." *Frontiers in neuroscience* 12 (2018).
- [6] Y.-F. Liu et al., "Slimmed optical neural networks with multiplexed neuron sets and a corresponding backpropagation training algorithm," *Intell*

Comput, vol. 3, p. 0070, Jan. 2024, doi: 10.34133/icomputing.0070.